Advanced Neural Networks: Vision, Sequences, and Generation

Lecture 30

CS 4880

Kirk Duran

One Network Shape Does Not Fit Every Input

A dense network accepts a list of features, but structure in the input matters.

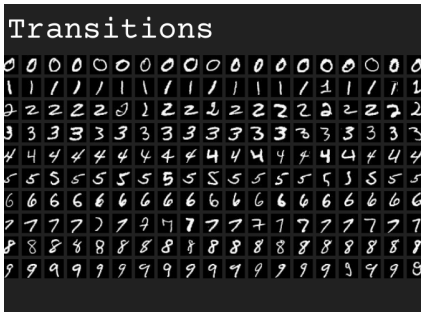
- In Lecture 29, every unit in a dense layer could connect to every feature of the previous layer.
- For an **image**, nearby pixels form meaningful patterns; flattening preserves pixel indices but gives a dense layer no built-in preference for local neighbors.
- For a **sequence**, the same observations in a different order may mean something entirely different.
- We can build useful assumptions into the **architecture**: locality for images and reusable state for sequences.

Key idea

Today's idea: architecture determines what relationships are easy for a model to learn from the same data.

MNIST Works; Natural Images Are Harder

A good result on one dataset does not establish that the model works on a different kind of image.



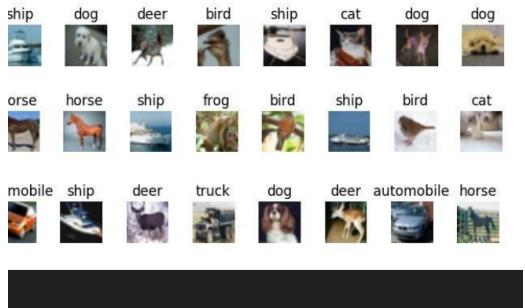
- MNIST: 28×28 grayscale digits; a dense network can perform well.
- CIFAR-10: 32×32 color objects with varied background, pose, and texture.
- A network's **inductive bias** is the structural preference built into its design.

What flattening does

A 32×32 RGB image becomes a vector of $32 \cdot 32 \cdot 3 = 3{,}072$ input numbers. Pixel positions still exist in the vector, but a generic dense layer does not explicitly assume nearby pixels are related.

CIFAR-10: A Test of Visual Generalization

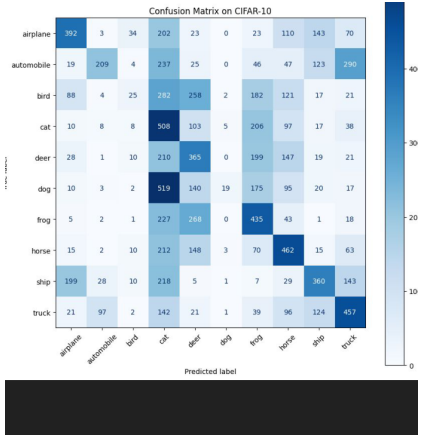
- 60,000 labeled RGB images, each 32×32 pixels.
- 50,000 training images and 10,000 test images.
- Ten classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.
- Small objects and variable backgrounds make classifying these pictures more challenging than centered digits.



The Original Dense-Network CIFAR-10 Result

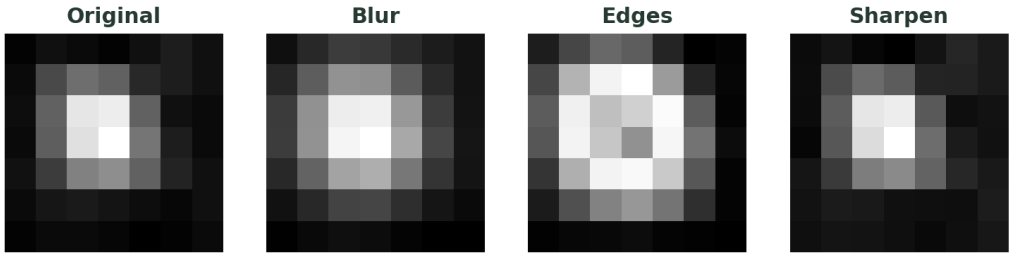
The earlier network was fitted, but its predictions on natural images were weak.

- The supplied experiment reports **32.32% test accuracy** after 20 epochs.
- On ten balanced classes, random guessing would average about 10%; the model learned something.
- Its confusion matrix reveals which categories it mixes up.
- The source does not provide all baseline settings, so treat later comparisons as *illustrative*, not a controlled ablation.



Before CNNs: Image Filters Were Designed by Hand

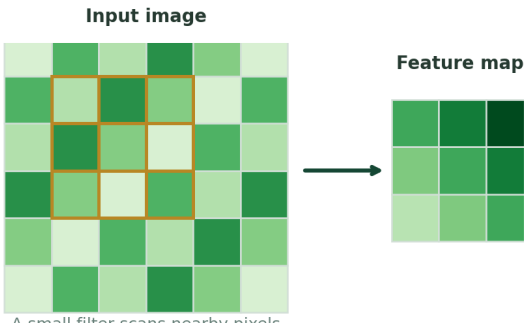
A small kernel can detect a local visual pattern by scanning across an image.



- Traditional image processing applies fixed **kernels**, such as blur or edge-detection filters.
- Each output location summarizes a small neighboring patch of pixels.
- These filters are useful, but designing the right ones requires assumptions and expertise.
- A **convolutional neural network (CNN)** learns the filter values from training data.

Convolution: Apply a Learned Filter Everywhere

One filter learns a visual pattern and searches for it across many image locations.



- A **receptive field** is the local input patch used for an output.
- Multiply filter and pixel values, then sum.
- Slide it to create a **feature map** of responses.
- Training changes the filter's weights.

Convolution: Name Each Part of the Calculation

The equation describes one filter response before an activation such as ReLU.

One filter at one location

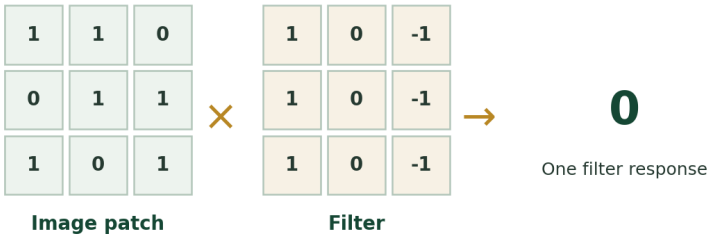
$$z_{u,v} = b + \sum_{i,j,c} K_{i,j,c}\, X_{u+i,v+j,c}.$$

X is the input; K contains one filter's weights; b is its bias. (u, v) is an output location; (i, j) are kernel offsets; c indexes input channels. z is the pre-activation response (stride 1, no padding). Libraries call this cross-correlation "convolution."

- K is shared across spatial locations; only the selected input patch changes.
- Summing over c combines all input channels into this one filter's output map.
- Another learned filter has its own weights and bias and produces another output channel.

A Tiny Convolution Calculation

The calculation is familiar multiplication and addition, repeated at many positions.



- Multiply the numbers in matching positions and add the nine products.
- Here the sum is zero: the chosen directional filter has no net response to this particular patch.
- Shift by one pixel and compute the next response; one filter produces a grid of responses whose size depends on padding and stride.

Why Convolution Is More Efficient Than Dense Layers

The same small set of weights is reused at every image location.

- **Locality:** early filters inspect nearby pixels, not every pixel in the image simultaneously.
- **Weight sharing:** a detector for an edge on the left also checks for that edge on the right.
- **Multiple filters:** distinct learned filters can respond to different textures, edges, or other patterns.
- **Stride** decides how far to move the filter each step; **padding** determines how image borders are handled.

Count the weights in a first convolutional layer

A 3×3 filter with 3 RGB channels has $3 \cdot 3 \cdot 3 = 27$ weights. With one bias per filter and 32 filters:

$$(3 \cdot 3 \cdot 3 + 1) \cdot 32 = 896 \text{ trainable parameters.}$$

A dense 3,072-input-to-32-unit layer would require $(3072 + 1) \cdot 32 = 98{,}336$ parameters. These layers do different work; the count shows the effect of weight sharing.

Track Shapes: Padding, Stride, and Pooling

Weights determine what is detected; padding and stride determine where responses are computed.

Output size along one spatial dimension

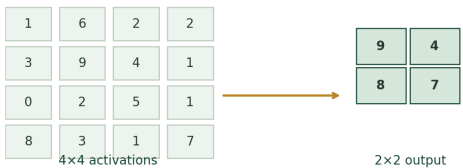
$$N_{\text{out}} = \left\lfloor \frac{N + 2p - k}{s} \right\rfloor + 1.$$

N: input size; k: kernel size; p: padding on each side; s: stride; $\lfloor \cdot \rfloor$: round down. This assumes dilation 1 and symmetric padding.

- A 32-wide input, 3-wide filter, no padding, stride 1 gives $32 - 3 + 1 = 30$ responses.
- Padding by 1 gives 32 responses. In our code, `padding='same'` with stride 1 preserves spatial size.
- A 2-wide pooling window with stride 2 and no padding maps 32 locations to 16.

Pooling: Keep Strong Signals, Shrink the Map

Pooling summarizes nearby responses so later layers process fewer locations.



- **Max pooling** keeps the strongest response in each local region.
- A 2×2 pool with stride 2 reduces width and height by about half.
- Small movements in a feature can still produce a similar pooled response.
- Some modern networks replace pooling with strided convolutions.

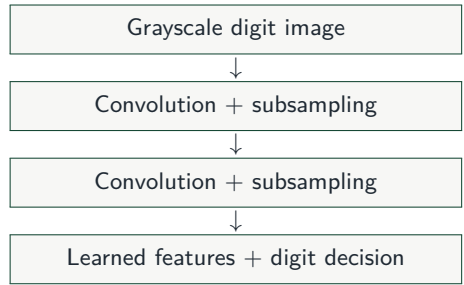
Key idea

Convolution extracts patterns; pooling compresses where those patterns were found.

LeNet-5: Learned Features for Handwritten Digits

Neural networks learned local image filters long before today's deep-learning boom.

- Yann LeCun and collaborators pioneered practical convolutional digit recognition; LeNet-5 was described in 1998.
- Repeated convolution and subsampling stages built increasingly useful representations of digits.
- The final classifier used these learned features to identify a number.
- The key architectural advantage was learning **local, shared features**, not hand-authoring every detector.

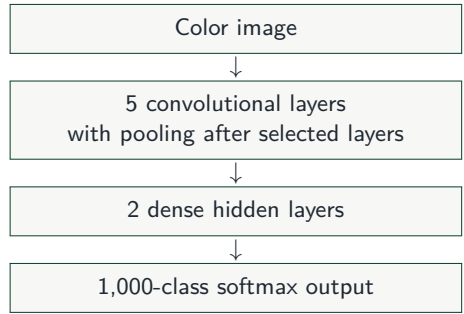


Simplified LeNet-5 flow; historical subsampling differs from our max-pooling example.

AlexNet: A Turning Point for CNNs

Larger datasets, GPUs, and trainable deep networks made CNNs more successful on natural images.

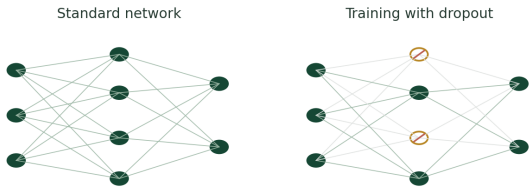
- AlexNet's 2012 ImageNet results demonstrated a major advance in large-scale visual recognition.
- Convolutional layers learned edges, textures, and object parts.
- ReLU activations, GPU training, data augmentation, and dropout helped make training practical.
- This was progress in the **full training system**, not a single clever filter.



Simplified AlexNet flow; original training used two GPUs.

Dropout: Train Without Relying on Every Unit

Randomly suppressing some activations during training can discourage fragile dependence on particular paths.



- During training, each selected activation is temporarily set to zero with some probability.
- The network must learn predictions that survive different sets of available units.
- During inference, dropout is switched off; frameworks account for scaling.

Dropout rate versus keeping a unit
For dropout rate $0 \leq r < 1$, each selected activation is kept with probability $1 - r$ during training. In *inverted dropout*, surviving values are scaled by $1/(1 - r)$ so each activation extquotesingle s expected value is unchanged.

Original CIFAR-10 Example: Load and Normalize

Preserve image geometry when preparing data for a CNN.

```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
from tensorflow.keras import Sequential
from tensorflow.keras.layers import (Input, Conv2D,
    MaxPooling2D, Dropout, Flatten, Dense)

(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train = x_train.astype("float32") / 255.0
x_test  = x_test.astype("float32") / 255.0
y_train = to_categorical(y_train, 10)
y_test  = to_categorical(y_test, 10)
```

Key idea

Every example remains a $32 \times 32 \times 3$ array. The test set is never passed to `fit`. The source lecture uses one-hot labels, so we retain categorical cross-entropy.

Original CIFAR-10 CNN: Convolutional Stages

Three convolution/pooling blocks progressively turn image pixels into compact feature maps.

```
model = Sequential([
    Input(shape=(32, 32, 3)),
    Conv2D(32, (3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.25),
    Conv2D(64, (3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.25),
    Conv2D(128, (3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.25),
])
```

Spatial scale across the network
The source's three 2×2 pooling stages reduce spatial size from 32×32 to 16×16, 8×8, then 4×4. The channel count increases from 32 to 64 to 128.

Original CIFAR-10 CNN: Classifier and Training

After extracting patterns, predict one of ten classes.

```
model.add(Flatten())
model.add(Dense(256, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(10, activation='softmax'))
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
history = model.fit(x_train, y_train, epochs=20,
                    validation_split=0.2, batch_size=128)
test_loss, test_acc = model.evaluate(x_test, y_test)
```

Key idea

The last 20% of training examples are held out for validation (40,000 fit / 10,000 validation); the official test set stays separate. `softmax` returns ten class probabilities summing to one. Historical accuracy on the next slide is not a promised rerun result.

CIFAR-10: What Changed with a CNN?

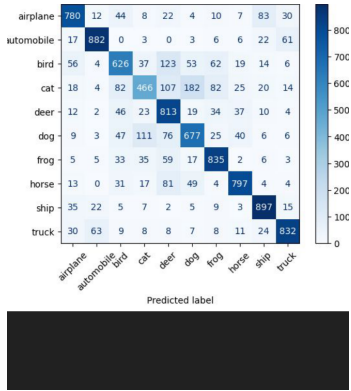
The source's CNN experiment reports stronger image classification than its earlier dense baseline.

Source lecture results

Dense baseline: 32.32% test accuracy.

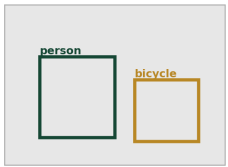
CNN example: 76.05% test accuracy.

- The CNN can reuse local visual features across an image.
- Incomplete source settings make this an illustrative comparison, not a controlled ablation.
- The confusion matrix helps identify remaining errors by category.



Classification Answers “What?”; Detection Adds “Where?”

A single class label cannot describe every object in a busy scene.



Classification

What is present?

Detection

What and where?

- **Classification:** choose a label for the image.
- **Detection:** identify objects and predict bounding boxes around them.
- Multiple objects may occur at different scales and overlap.
- Robotics needs both object identity and approximate location.

From Region Proposals to YOLO

One-stage detectors predict objects directly rather than classifying a long list of candidate crops.

Two-stage detection

Propose candidate regions, extract features, and classify/refine candidate boxes. Multiple steps can increase compute cost, though modern systems can be accurate and efficient.

YOLO: You Only Look Once

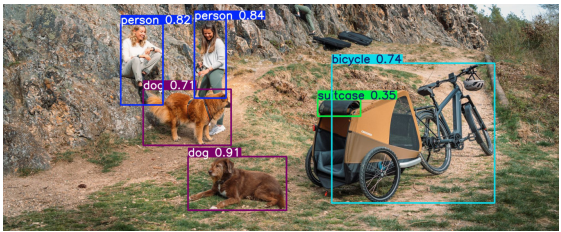
A neural network predicts object locations and categories in one forward pass through the image. The original YOLO used a spatial grid; later versions have evolved beyond this exact design.

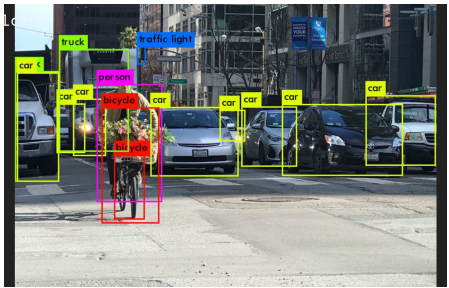
A predicted detection

A box can be encoded as (x, y, w, h): center coordinates x, y, width w, height h, with pixel or normalized units specified by the implementation. Detectors also output class scores and may output a separate objectness score; the exact encoding depends on the model.

Original YOLO Examples: Multiple Objects at Once

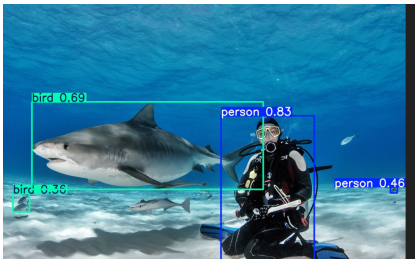
A detector can label several objects in the same image, each with a separate region.





Reading YOLO Outputs Critically

Fast object detection is useful, but predictions are not automatically correct.



- A confidence score describes model output, not a promise that a label is correct.
- Small, occluded, or overlapping objects can be difficult to detect and localize.
- A wrong object label or misplaced box can directly affect a robot's planned behavior.
- Different detectors trade off latency, memory, localization, and category accuracy.

How to compare a predicted box with a labeled box

$\mathrm{IoU}(B_p, B_t) = \dfrac{\mathrm{area}(B_p \cap B_t)}{\mathrm{area}(B_p \cup B_t)}$, where B_p is the predicted box and B_t the labeled box. Intersection is shared area; union counts all covered area once. IoU measures location overlap, not class correctness.

Images Have Position; Sequences Have Order

Treating a sequence as an unordered bag loses information about what happened first.

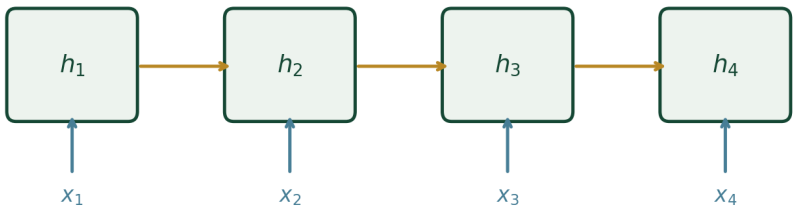
- **Language:** "dog bites man" and "man bites dog" contain the same words in a different order.
- **Video:** movement depends on the ordering of frames.
- **Audio:** sound unfolds over time; the timing of phonemes matters.
- **Time series:** future sensor or price values may depend on earlier observations.

Key idea

A recurrent neural network carries a changing **hidden state** from one time step to the next. It is a learned summary of earlier inputs, not a perfect memory.

Recurrent Networks: One Operation, Reused Over Time

A recurrent unit processes the next input together with a summary of the past.



- The **same learned weights** are applied at every time step.
- The updated hidden state is passed forward to the next time step.
- **Unrolling** a recurrent network draws the operations across time, without creating separate independent models.

What Does the RNN Update Actually Mean?

New memory is computed from new evidence and old memory.

1. Read the current input x_t (for example, today's sensor measurement).
2. Combine it with the hidden state from the previous time step h_{t-1}.
3. Apply a nonlinear activation to form the new state h_t.
4. Use the state to predict the current or future output.

A simple recurrent update

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b), \qquad \widehat{y}_t = g(W_y h_t + c).$$

t indexes time; x_t is input and h_t the hidden state ($h_0 = 0$ here). W_x, W_h, W_y are learned input, recurrent, and output weight matrices; b, c are biases. $\widehat{y}_t$ is the prediction; g is the output activation (identity for regression). $\tanh$ acts entry by entry.

Build a Reproducible Sequence Before Fitting

Our runnable example uses a synthetic sensor series, not an unexplained stock-data variable.

```
import numpy as np
rng = np.random.default_rng(4880)
t = np.arange(600)
series = np.sin(t / 15) + 0.1 * rng.normal(size=600)
train_end, val_end, window = 360, 480, 20
mean, scale = series[:train_end].mean(), series[:train_end].std()
values = ((series - mean) / scale).astype('float32')
target_time = np.arange(window, len(values))
X = np.stack([values[j-window:j] for j in target_time])[..., None]
y = values[target_time]
```

Key idea

Each input contains the previous 20 observations, and its target is the next value. Scaling uses training history only. The AAPL plots that follow are supplied historical illustrations, not outputs from this sensor example.

Split Windows by the Time of Their Target

A forecast may use observed history, but it must not use future values.

```
train = target_time < train_end
val = (target_time >= train_end) & (target_time < val_end)
test = target_time >= val_end
X_train, y_train = X[train], y[train]
X_val, y_val = X[val], y[val]
X_test, y_test = X[test], y[test]
models = {}
```

- Shapes: training $(340, 20, 1)$; validation $(120, 20, 1)$; test $(120, 20, 1)$. Axes are samples, time steps, features.
- Validation/test inputs may include earlier observed values. Every input ends *before* its own target: one-step forecasting, not a 120-step forecast from one origin.
- Each window starts with a fresh hidden state; the RNN carries state *within* that window.

Original Python Example: SimpleRNN for a Time Series

Use a window of past values to predict a numerical next value.

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, SimpleRNN, Dense

# X_train shape: (samples, time_steps, features)
model = Sequential([
    Input(shape=(X_train.shape[1], X_train.shape[2])),
    SimpleRNN(64, activation='tanh'),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse')
models['SimpleRNN'] = model
```

Key idea

This example consumes the sensor windows just defined. We use 64 hidden units in all three recurrent models; the source SimpleRNN used 50. With `return_sequences=False` (default), only the final hidden output feeds `Dense(1)`.

Why Ordinary RNNs Struggle with Long Histories

The learning signal has to travel backward across many repeated time steps.

- **Vanishing gradients:** repeated calculations can shrink learning signals until early time steps have little influence.
- **Exploding gradients:** repeated calculations can amplify them into unstable updates.
- The model may fail to learn dependencies between distant events.
- Gradient clipping can help exploding gradients; gating helps preserve useful longer-term information.



Why depth through time matters

Backpropagation through time repeatedly applies local derivatives. A long product of factors smaller than one can vanish; factors larger than one can explode. This is the gradient problem, not simply a missing software option.

LSTM: Learn What to Retain and What to Forget

Gates give the network a more controlled path for carrying information over time.

- A **Long Short-Term Memory (LSTM)** cell maintains a cell state in addition to a hidden output.
- The **forget gate** decides how much previous state to retain.
- The **input gate** decides what new information to write.
- The **output gate** decides what portion of stored information to expose.
- This improves learning of many long-range dependencies, though it does not guarantee perfect memory.

Cell-state update: retention plus new content

$$c_t = f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t.$$

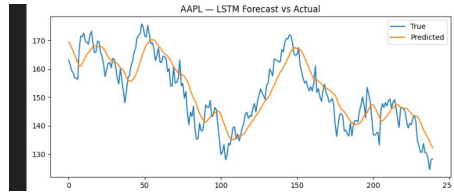
t indexes time; c_t is cell memory and $\widetilde{c}_t$ proposed new content. f_t, i_t are learned forget/input gate vectors with entries in $[0, 1]$; $\odot$ multiplies corresponding entries. This is the memory update; the hidden output also uses an output gate.

Original LSTM Example: Replace the Recurrent Layer

Keep the same input windows and prediction target; change the memory mechanism.

```
from tensorflow.keras.layers import LSTM
from tensorflow.keras import Sequential

model = Sequential([
    Input(shape=X_train.shape[1:]),
    LSTM(64),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse')
models['LSTM'] = model
```



Historical source: LSTM on AAPL prices; not this sensor run.

GRU: A Smaller Gated Recurrent Unit

GRUs use a simpler arrangement of gates to manage a changing hidden state.

- A **Gated Recurrent Unit (GRU)** combines information retention and updating without a separate LSTM-style cell state.
- An **update gate** controls how much of the old state to preserve versus overwrite.
- A **reset gate** controls how much past information influences a new candidate state.
- GRUs often have fewer parameters than similarly sized LSTMs, but performance depends on the task and settings.

A simplified GRU-style interpolation

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \widetilde{h}_t.$$

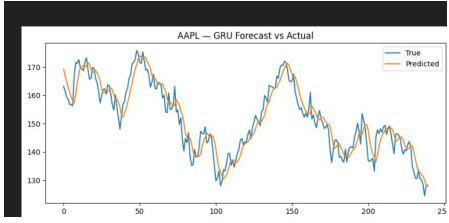
h_t is hidden state at time t; $\widetilde{h}_t$ is the candidate. The update gate z_t has entries in $[0, 1]$: here 1 retains old state and 0 replaces it. 1 means an all-ones vector; $\odot$ is entrywise multiplication. The reset gate helps form the candidate (not shown).

Original GRU Example: Another Memory Mechanism

We can hold the surrounding prediction setup fixed and change the recurrent unit.

```
from tensorflow.keras.layers import GRU
from tensorflow.keras import Sequential

model = Sequential([
    Input(shape=X_train.shape[1:]),
    GRU(64),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse')
models['GRU'] = model
```



Historical source: GRU on AAPL prices; not this sensor run.

Train Equally; Choose on Validation; Test Once

```
val_mse = {}
for name, model in models.items():
    model.fit(X_train, y_train, epochs=10, batch_size=32,
              validation_data=(X_val, y_val), shuffle=False, verbose=0)
    val_mse[name] = model.evaluate(X_val, y_val, verbose=0)
best = models[min(val_mse, key=val_mse.get)]
pred = best.predict(X_test, verbose=0).ravel()
rmse = scale * np.sqrt(np.mean((pred - y_test)**2))
naive = scale * np.sqrt(np.mean((X_test[:, -1, 0] - y_test)**2))
print('test RMSE:', rmse, 'last-value RMSE:', naive)
```

What the metric means

$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{j=1}^{n}(\widehat{y}_j - y_j)^2}$: root mean squared error across n test targets. y_j is observed and $\widehat{y}_j$ predicted. Multiplying by the training standard deviation `scale` restores sensor units.

Choosing a Sequence Model Requires a Fair Test

Different-looking prediction plots do not tell us which model generalizes best.

- Give the SimpleRNN, LSTM, and GRU the same historical input windows and target definition.
- Split by **time**; random window splits can train on future targets and near-duplicate histories.
- Fit preprocessing only on past training observations; apply unchanged to later observations.
- Compare MAE/RMSE and a **naive last-value forecast** before claiming an architecture is useful.
- Equal hidden width does not mean equal parameter count. Report model capacity and training budget alongside accuracy.

Key idea

The new architecture helps only if it performs better under an honest, controlled evaluation of future data.

Discriminative versus Generative Learning

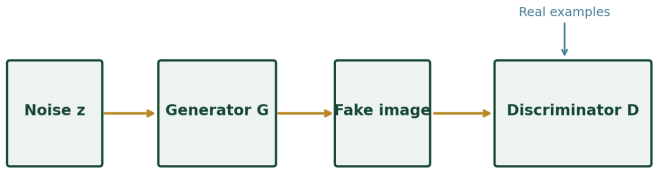
Not every model learns to assign a label; some learn to produce new examples.

Discriminative question	Generative question
Given an image, **which label** best describes it?	Given a learned training distribution, **what new image** could plausibly come from it?
Example: classify a digit as 7.	Example: generate a digit-like image from noise.

- A **Generative Adversarial Network (GAN)** uses two networks trained in competition.
- The generator makes artificial examples; the discriminator learns to distinguish generated examples from real ones.

A GAN Trains Two Networks Against Each Other

One model learns to imitate the training data; the other provides a changing quality signal.



- **Generator** G**:** maps a random latent input z to a candidate image $G(z)$.
- **Discriminator** D**:** returns a logit (unbounded score) for how real an input looks.
- Training alternates updates: improve discrimination, then improve generation using the discriminator's feedback.

Original GAN Generator: Turn Noise into Images

A generator expands a small random vector into a 28×28 grayscale image.

```
from tensorflow.keras import layers

def build_generator():
    return tf.keras.Sequential([
        layers.Input(shape=(100,)),
        layers.Dense(7*7*256, use_bias=False),
        layers.BatchNormalization(), layers.LeakyReLU(),
        layers.Reshape((7, 7, 256)),
        layers.Conv2DTranspose(128, (5,5), strides=(1,1),
                               padding='same', use_bias=False),
        layers.BatchNormalization(), layers.LeakyReLU(),
        layers.Conv2DTranspose(64, (5,5), strides=(2,2),
                               padding='same', use_bias=False),
        layers.BatchNormalization(), layers.LeakyReLU(),
        layers.Conv2DTranspose(1, (5,5), strides=(2,2),
                               padding='same', use_bias=False,
                               activation='tanh')
    ])
```

Original GAN Discriminator: Evaluate Real or Fake

The discriminator compresses an image into a real-versus-generated score.

```
def build_discriminator():
    return tf.keras.Sequential([
        layers.Input(shape=(28, 28, 1)),
        layers.Conv2D(64, (5,5), strides=(2,2), padding='same'),
        layers.LeakyReLU(),
        layers.Dropout(0.3),
        layers.Conv2D(128, (5,5), strides=(2,2), padding='same'),
        layers.LeakyReLU(),
        layers.Dropout(0.3),
        layers.Flatten(),
        layers.Dense(1)
    ])
```

Key idea

The final `Dense(1)` returns a **logit** (unbounded score), not a probability. The training loss must be configured consistently, such as `BinaryCrossentropy(from_logits=True)`.

How GAN Training Connects the Two Networks

Defining two architectures does not yet train a GAN.

1. Scale real MNIST images to $[-1, 1]$: $(x - 127.5)/127.5$ for raw pixel value $x \in [0, 255]$. This matches the generator's `tanh` output.
2. Draw noise vectors of length 100. Produce fake images with G; score both real and fake images with D.
3. Update **only** D**'s weights** to label real images 1 and fake images 0.
4. Update **only** G**'s weights** to make D label fake images 1. Backpropagate *through* D to reach G.

A common non-saturating generator loss

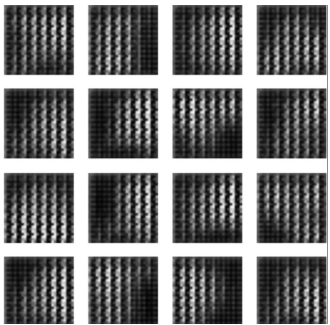
$L_D = \mathrm{BCE}(1, D(x)) + \mathrm{BCE}(0, D(G(z))), \quad L_G = \mathrm{BCE}(1, D(G(z)))$.

x is a real image; z is noise; G, D are generator/discriminator. BCE is binary cross-entropy, averaged over a minibatch; labels come first. Here D returns logits, so use `from_logits=True`. Separate optimizers update the two parameter sets.

GAN Training: Images Change Over Epochs

Supplied snapshots from one historical run show emerging digit structure; improvement is not guaranteed or monotonic.

Epoch 1



Epoch 7

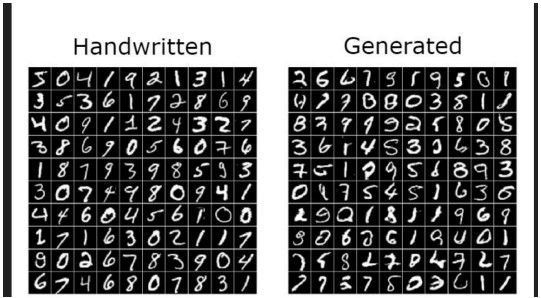


Epoch 19



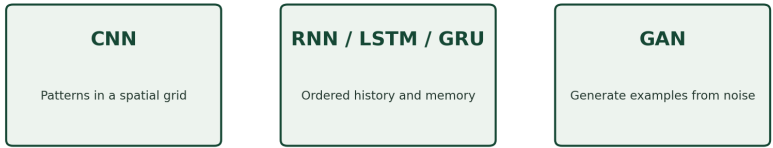
Generated Samples, Progress, and Failure Modes

Believable examples are the goal, but stable adversarial training is difficult.



- The generator can learn digit-like structure without being handed a unique target image for each noise vector.
- **Mode collapse:** the generator repeatedly produces a narrow set of examples.
- **Instability:** alternating adversarial optimization can oscillate or fail to converge.
- Visual quality is only one dimension of evaluation; diversity also matters.

Match the Architecture to the Structure of the Problem



- **CNNs:** learn reusable local filters for images and spatial structure.
- **RNN/LSTM/GRU:** share weights across time and carry learned state for sequences.
- **GANs:** train a generator with feedback from a competing discriminator.
- More architectural complexity does not replace good data, evaluation, or deployment constraints.

Key idea

Next: unsupervised learning—discovering groups and useful representations without given class labels.

References and Reproducible Checks

- Supplied Lecture 30 image archive: original result plots are historical, not regenerated benchmarks.
- CIFAR-10 dataset description; Keras Conv2D; Keras GRU.
- LeCun et al. (1998), LeNet-5; Krizhevsky et al. (2012), AlexNet.
- TensorFlow DCGAN tutorial: compatible generator/discriminator, normalization, and adversarial losses.
- Companion `assets/lecture-30/verify-examples.py`: checks convolution/pooling arithmetic, CNN shapes and a training update, the full displayed sensor-model runs, and generator/discriminator gradients.

Key idea

The CNN's full 20-epoch CIFAR-10 run and historical GAN training curves are not reproduced by the lightweight verification script.